Лабораторна робота №8. Pressable, зображення і хуки в React Native

Мета роботи

Навчитися обробляти натискання через `Pressable`, змінювати вміст екрана за допомогою стану, підключати локальні зображення та пристосовувати інтерфейс до системної світлої або темної теми. Закріпити використання TypeScript для стану, властивостей компонентів і функцій-обробників.

Що має працювати наприкінці

Продовжте застосунок **Little Lemon** із лабораторних робіт №6–7. Реалізуйте три завдання:

1. Додайте до форми кнопку на основі `Pressable`. Після коректного заповнення полів вона відкриває навчальний екран привітання; із нього можна повернутися до форми.
2. Покажіть на екрані привітання локальний логотип Little Lemon без спотворення пропорцій.
3. Налаштуйте автоматичне перемикання кольорів усього поточного інтерфейсу відповідно до системної теми.

Шапка, підвал, безпечні відступи й робота з клавіатурою залишаються частинами застосунку. Компоненти пишіть у файлах `.tsx`, спільні типи й палітри — у `.ts`. Збережіть також файли списків із №7: вони знадобляться далі, хоча в цій роботі основну увагу приділяємо формі та привітанню.

Це навчальна імітація переходу після заповнення форми, а не справжній вхід до облікового запису. Сервер не перевіряє дані, користувач не отримує прав доступу. Використовуйте вигадані email і пароль. Не зберігайте пароль у файлі, сховищі чи консолі та не передавайте його на екран привітання.

Підготовка

Відкрийте свій проєкт із №7. Збережіть поточну версію у Git або зробіть резервну копію без `node_modules`. У терміналі в папці проєкту виконайте:

```
npm install
npx expo install react-native-safe-area-context expo-system-ui
npx expo start
```

`npm install` установлює залежності, описані у проєкті. `npx expo install` добирає версії додаткових пакетів, сумісні з вашим Expo SDK. Бібліотека `react-native-safe-area-context` потрібна для безпечних меж екрана; якщо вона вже встановлена, повторно створювати оболонку не треба. `expo-system-ui` забезпечує підтримку налаштування системної теми у власній Android-збірці. `Pressable`, `Image`, `useColorScheme` та `useWindowDimensions` входять до `react-native` й не потребують окремих пакетів.

Не оновлюйте Expo або React Native лише заради цієї роботи. Продовжуйте використовувати сумісний набір залежностей свого проєкту.

Перевірте, що форма з №7 відкривається, email і пароль мають незалежні значення, пароль маскується, а клавіатура не перекриває нижню частину форми.

Якщо проєкту ще немає, спочатку створіть основу:

```
npx create-expo-app@latest little-lemon --template blank-typescript
cd little-lemon
npx expo install react-native-safe-area-context expo-system-ui
```

У такому проєкті відтворіть шапку, підвал, безпечну оболонку та форму з №6–7. У шаблоні `blank-typescript` основним файлом є `App.tsx`; установлювати бібліотеку навігації для цієї роботи не потрібно.

Теоретичні відомості

Натискання і компонент Pressable

Звичайний `Text` показує напис, а `Pressable` дозволяє перетворити блок із цим написом на інтерактивний елемент. Наприклад, структура кнопки — це `Pressable` зовні та `Text` усередині. Відступи й фон належать контейнеру, а колір та розмір напису — тексту.

Основні властивості:

Властивість	Для чого потрібна
`onPress`	Передає функцію, яку потрібно виконати після натискання.
`style`	Приймає стиль або функцію, що обчислює стиль із поточного `pressed`.
`disabled`	Вимикає обробку натискань. Самостійно не пояснює користувачу причину недоступності.
`hitSlop`	Розширює зону початку дотику, не збільшуючи видимий елемент. Зона обмежена батьківським контейнером.
`accessibilityRole="button"`	Повідомляє екранному читачу, що елемент є кнопкою.

Правильний запис обробника:

```
<Pressable onPress={handleContinue}>
  <Text>Продовжити</Text>
</Pressable>
```

Тут у `onPress` передається **сама функція**. Запис `onPress={handleContinue()}` викликав би її під час обчислення JSX, ще до дії користувача. Якщо треба передати аргумент, використайте функцію-обгортку: `onPress={() => handleSelect('menu')}`.

`pressed` означає, що користувач зараз утримує елемент. Це короткочасний візуальний стан, а не ознака виконаного входу чи вибраної сторінки. Для збереження результату дії потрібен власний стан.

Стан, повторне відображення й умовний вміст

Змінна, оголошена звичайним `let`, сама по собі не змушує React оновити екран. `useState` зберігає значення між відображеннями компонента, а його setter запитує нове відображення:

```
const [showDetails, setShowDetails] = useState(false);

// Усередині return компонента:
<Pressable onPress={() => setShowDetails(previous => !previous)}>
  <Text>{showDetails ? 'Приховати опис' : 'Показати опис'}</Text>
</Pressable>
```

`false` задає початкове значення. TypeScript виводить із нього тип `boolean`. `previous => !previous` отримує попереднє значення та повертає протилежне. Ця форма доречна, коли новий стан залежить від старого.

Стан належить конкретному компоненту. Якщо компонент форми прибрати з дерева, його локальні поля буде скинуто. Якщо дані потрібні і формі, і привітанню, їх зберігають вище — у спільному батьківському компоненті — та передають через `props`.

У цій роботі достатньо зберігати email навчальної форми або відсутність такого значення:

```
const [demoEmail, setDemoEmail] = useState<string | null>(null);
```

`string | null` — об'єднання двох типів. `null` означає «навчальний перехід ще не виконано», а рядок — email, який треба показати у привітанні. Явний тип потрібен, оскільки початкового `null` недостатньо, щоб описати майбутнє рядкове значення.

Як TypeScript описує взаємодію компонентів

Дочірній компонент не повинен сам змінювати стан батьківського. Натомість він отримує функцію через `props` і викликає її, коли дію завершено:

```
type ContinueHandler = (email: string) => void;
```

Цей тип описує функцію, яка отримує один рядок і не повертає значення, потрібного виклику. `void` не означає, що функція нічого не робить: вона може оновлювати стан.

Порівняйте опис властивостей у JavaScript і TypeScript:

```
// JavaScript: форму властивостей потрібно знати з використання компонента.
function Greeting({ email }) {
  return <Text>Вітаємо, {email}!</Text>;
}
```

```
// TypeScript: контракт властивостей записано явно.
type GreetingProps = { email: string };

function Greeting({ email }: GreetingProps) {
  return <Text>Вітаємо, {email}!</Text>;
}
```

Обидва приклади потребують імпорту `Text` із `react-native`. TypeScript дозволить раніше помітити випадкову передачу числа чи пропущену властивість, але не перевірить, чи існує email і чи правильний пароль. Дані, які вводить користувач, потрібно перевіряти під час роботи програми.

Локальні зображення

`Image` відображає зображення, а `source` визначає його джерело. Локальний файл підключають статичним шляхом:

```
<Image source={require('../assets/images/logo.png')} />
```

Шлях рахується **від файлу компонента**, а не від кореня всього проєкту. `..` означає перехід на одну папку вище. Якщо компонент лежить у `screens/`, а `assets/` — у корені, потрібен саме такий початок шляху. Регістр літер має збігатися з назвою файла.

У цьому випадку `require` повідомляє збирачу Metro про локальний ресурс. Це не причина переводити файли застосунку з `import` / `export` на CommonJS. Шлях має бути відомим під час збирання: не складайте його з введеного користувачем тексту.

Видимий розмір і спосіб масштабування задають окремо. `resizeMode="contain"` показує зображення повністю зі збереженням пропорцій; навколо можуть залишатися вільні ділянки. `cover` заповнює рамку зі збереженням пропорцій, але може обрізати краї. Для логотипа зазвичай підходить `contain`.

Системна тема і правила хуків

`useState` імпортується з `react`, а `useColorScheme` — із `react-native`. Останній повертає поточну колірну схему й реагує на її зміни. Для цієї роботи використайте світлу палітру, якщо значення не дорівнює `'dark'`:

```
const colorScheme = useColorScheme();
const isDark = colorScheme === 'dark';
```

Результатом може бути `'light'`, `'dark'` або `null`, якщо відповідний нативний модуль недоступний. За наведеною умовою `null` дасть світлий варіант. Не копіюйте системну тему в

`useState`: це створить зайвий стан, який доведеться синхронізувати.

Викликайте ці хуки на верхньому рівні функції компонента, до умовного `return`. Не викликайте їх усередині `onPress`, циклу чи гілки `if`. Усередині обробника можна викликати вже отриманий setter, наприклад `setDemoEmail(null)`.

Зміна фону не змінює автоматично колір усіх написів. Для темної теми треба узгоджено задати також текст, поля введення, підказки, межі й повідомлення про помилки. Базові розміри зберігайте у `StyleSheet`, а поточні кольори передавайте через масив стилів:

```
<Text style={[styles.message, { color: colors.text }]}>
  Ласкаво просимо!
</Text>
```

Якщо кілька стилів задають ту саму властивість, пізніший перекриває попередній. Тому колір із поточної палітри тут має перевагу над кольором у `styles.message`.

Виконання роботи

Крок 1. Підготуйте файли й спільні кольори

Збережіть наявні компоненти й використайте таку структуру для потрібної частини проєкту:

```
App.tsx
app.json
assets/
  images/
    logo.png
components/
  LittleLemonHeader.tsx
  LittleLemonFooter.tsx
screens/
  LoginScreen.tsx
  WelcomeScreen.tsx
theme/
  palette.ts
```

Папки списків і даних із №7 залиште на місці. Якщо ви використовували інші назви компонентів або іменовані експорти, узгодьте з ними імпорти; не створюйте другу копію того самого екрана.

Створіть `theme/palette.ts`:

```
export type Palette = {
  background: string;
  surface: string;
  text: string;
  muted: string;
  border: string;
  accent: string;
  onAccent: string;
```

```
  error: string;
};

export const lightColors: Palette = {
  background: '#FFFFFF',
  surface: '#EDEFEE',
  text: '#25332E',
  muted: '#46584F',
  border: '#697B70',
  accent: '#EE9972',
  onAccent: '#222222',
  error: '#B3261E',
};

export const darkColors: Palette = {
  background: '#121815',
  surface: '#23312A',
  text: '#F2F5F3',
  muted: '#B4C4BA',
  border: '#81958A',
  accent: '#EE9972',
  onAccent: '#222222',
  error: '#FFB4AB',
};
```

Назви описують призначення кольору: `surface` — фон картки чи поля, `muted` — другорядний текст, `onAccent` — напис на акцентному фоні. Обидві палітри мають однакову форму, тому компоненти можуть використовувати будь-яку з них без зміни JSX. Можна дібрати власні кольори, зберігши читабельність.

Завдання 1. Додайте Pressable і навчальний перехід

1.1. Задайте контракт компонентів

До властивостей `LoginScreen` додайте палітру й функцію, яку буде викликано після локальної перевірки:

```
import type { Palette } from '../theme/palette';

type LoginScreenProps = {
  colors: Palette;
  onContinue: (email: string) => void;
};

// Заголовок вашої функції компонента:
// export default function LoginScreen({ colors, onContinue }: LoginScreenProps) {
//   Тут залишаються стани, обробники, return і розмітка форми.
// }
```

Це опис властивостей, а не повний файл. Збережіть два керовані поля з №7 та надайте їм кольори через `colors`. Додайте стан повідомлення про помилку:

```
const [error, setError] = useState<string | null>(null);
```

На початку `error` дорівнює `null`, тобто повідомлення відсутнє. Рядок у цьому стані означатиме текст, який потрібно показати під полями.

1.2. Обробіть дію форми

Імпортуйте `Keyboard` із `react-native`. Усередині `LoginScreen`, після оголошення станів `email`, `password` і `error`, додайте обробник:

```
function handleContinue() {
  const normalizedEmail = email.trim();
  const emailLooksValid = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(normalizedEmail);

  if (!emailLooksValid) {
    setError('Введіть email у форматі name@example.com.');
    return;
  }

  if (password.length === 0) {
    setError('Введіть навчальний пароль.');
    return;
  }

  setError(null);
  Keyboard.dismiss();
  setPassword('');
  onContinue(normalizedEmail);
}
```

`trim()` прибирає пробіли на краях email. Перевіряйте саме локальну змінну `normalizedEmail`, а не розраховуйте, що `setEmail` одразу змінить значення `email` у поточному обробнику. Регулярний вираз перевіряє лише приблизний формат, не існування скриньки.

`return` у кожній помилковій гілці завершує обробник: перехід не виконається. Пароль перевіряється лише як непорожній рядок, без обрізання пробілів. Це умова навчальної вправи, а не політика безпеки справжнього сервісу. `Keyboard.dismiss()` прибирає клавіатуру. У callback передається тільки email; пароль очищається.

1.3. Створіть кнопку і повідомлення

Після полів, усередині прокручуваного вмісту форми, розмістіть:

```
{error !== null && (
  <Text style={[styles.message, { color: colors.error }]}>
    {error}
  </Text>
)}

<Pressable
  accessibilityRole="button"
```

```
  onPress={handleContinue}
  style={({ pressed }) => [
    styles.button,
    { backgroundColor: colors.accent },
    pressed && styles.buttonPressed,
  ]}
>
  <Text style={[styles.buttonText, { color: colors.onAccent }]}>
    Увійти (демо)
  </Text>
</Pressable>
```

Додайте `Pressable` до імпорту з `react-native`. Оператор `&&` показує повідомлення лише тоді, коли умова істинна. Аналогічно стиль `buttonPressed` застосовується тільки під час утримання кнопки.

Додайте ці властивості **до наявного** `StyleSheet.create` форми, не замінюючи стилі полів і контейнерів:

```
message: { fontSize: 16, lineHeight: 24, marginVertical: 12 },
button: {
  minHeight: 48,
  paddingVertical: 12,
  paddingHorizontal: 20,
  borderRadius: 8,
  alignItems: 'center',
  justifyContent: 'center',
  marginTop: 16,
},
buttonPressed: { opacity: 0.75 },
buttonText: { fontSize: 18, fontWeight: '600', textAlign: 'center' },
```

Це властивості об'єкта стилів, а не окремі команди. `minHeight`, на відміну від жорсткого `height`, дозволяє кнопці збільшитися, якщо напис переноситься або користувач збільшив шрифт. На `ScrollView` форми залиште `keyboardShouldPersistTaps="handled"`, щоб кнопка могла обробити дотик із відкритою клавіатурою.

У цій реалізації кнопка залишається доступною і для порожньої форми: натискання пояснює помилку. Не додавайте `disabled` бездумно, інакше користувач не зможе отримати це пояснення.

1.4. Пов'яжіть форму з привітанням

У `App.tsx` імпортуйте `useState` із `react`, обидва екрани та `lightColors` із `./theme/palette`. Усередині `App`, перед `return`, додайте:

```
const [demoEmail, setDemoEmail] = useState<string | null>(null);
const colors = lightColors;
```

Поки що використовується світла палітра; системну тему додамо у завданні 3. У середній області оболонки розмістіть умовний вміст:

```
<View style={styles.content}>
  {demoEmail === null ? (
    <LoginScreen colors={colors} onContinue={setDemoEmail} />
  ) : (
    <WelcomeScreen
      colors={colors}
      email={demoEmail}
      onExit={() => setDemoEmail(null)}
    />
  )}
</View>
```

Імпортуйте `View` із `react-native`, задайте `content: { flex: 1 }`. Це фрагмент усередині наявної безпечної оболонки: шапка стоїть над ним, підвал — під ним. Не додавайте спільний зовнішній `ScrollView`: форма вже має власне прокручування.

Тернарний оператор `умова ? варіант1 : варіант2` вибирає лише один екран. `setDemoEmail` підходить як callback: він приймає рядок, який передає форма. Для виходу функція-обгортка передає `null`.

У `WelcomeScreen.tsx` опишіть властивості:

```
import type { Palette } from '../theme/palette';

type WelcomeScreenProps = {
  colors: Palette;
  email: string;
  onExit: () => void;
};
```

Змініть заголовок функції на `export default function WelcomeScreen({ colors, email, onExit }: WelcomeScreenProps)`. Залиште опис закладу, додайте напис «Дані перевірено локально. Відкрито навчальний екран для …» із `{email}` та кнопку `Pressable` з `onPress={onExit}` і написом «Повернутися до форми». Оформіть її за тим самим принципом, що й кнопку продовження; стилі визначте в цьому файлі або винесіть спільну кнопку в окремий компонент.

Якщо на привітанні збереглося поле імені з №7, не вилучайте його: воно може залишатися незалежним від email.

Після повернення форма монтується заново, тому її локальні поля порожні. Після повного перезавантаження застосунку `demoEmail` теж знову дорівнює `null`. Це очікувано: постійного зберігання в цій роботі немає. Перемикання компонентів через стан не створює стек навігації та історію переходів.

Завдання 2. Додайте локальний логотип

1. Скопіюйте наданий поруч із завданням файл `assets/logo.png` до **свого проєкту** як `assets/images/logo.png`. Або використайте власний навчальний логотип PNG/JPEG.
2. У `screens/WelcomeScreen.tsx` додайте `Image` до імпорту з `react-native`.

3. Перед текстом привітання розмістіть зображення:

```
<Image
  source={require('../assets/images/logo.png')}
  style={styles.logo}
  resizeMode="contain"
  accessible={true}
  accessibilityLabel="Логотип Little Lemon"
/>
```

У стилі `logo` задайте, наприклад, `width: 120`, `height: 140`, `alignSelf: 'center'` і `marginBottom: 16`. Це розмір рамки зображення, а не зміна самого файла. Не задавайте іншого `resizeMode` одночасно в стилі: достатньо одного значення у властивості компонента.

Тимчасово порівняйте `contain` і `cover` в однаковій рамці. Поясніть різницю, після чого залиште `contain`. Перевірте, що логотип завантажується і без інтернету після встановлення або завантаження застосунку. Сам сервер розробки Expo при цьому може потребувати з'єднання з комп'ютером.

Інформативному логотипу надано підпис для екранного читача. Якщо зображення лише декоративне й повторює сусідню назву, можна натомість задати `accessible={false}`, щоб уникнути зайвого повторення. Підпис не є видимим текстом під зображенням.

Завдання 3. Реалізуйте системну світлу й темну тему

3.1. Дозвольте автоматичний режим у Expo

Відкрийте `app.json`. Усередині **наявного** об'єкта `expo` додайте або змініть властивість:

```
"userInterfaceStyle": "automatic"
```

Це один запис конфігурації, а не весь файл JSON. Збережіть назву, `slug`, іконки та інші налаштування. Якщо після цього запису є інші властивості, відокремте їх комою. Не додавайте другий об'єкт `expo`.

Після зміни конфігурації перезапустіть сервер Expo й повністю перезавантажте застосунок. Власну development build потрібно перебудувати, якщо змінювалися нативні налаштування чи додавався `expo-system-ui`. Звичайне збереження `.tsx` не застосовує зміни до вже зібраної нативної оболонки.

3.2. Обчислюйте палітру в App

Додайте `useColorScheme` до імпорту з `react-native`, а `darkColors` — до імпорту палітр. Усередині `App` замініть рядок `const colors = lightColors` на:

```
const colorScheme = useColorScheme();
const isDark = colorScheme === 'dark';
const colors = isDark ? darkColors : lightColors;
```

Ці рядки повинні виконуватися до `return`, незалежно від того, який екран відкритий. При зміні системної теми React отримає нове значення хука, знову обчислить компонент і передасть іншу палітру дочірнім компонентам. Для цього не потрібні окремий таймер, обробник натискання або `useEffect`.

Застосуйте `colors.background` до кореневого `SafeAreaView` із `react-native-safe-area-context`. Збережіть `SafeAreaProvider` у корені. Не замінюйте його відступи фіксованим `marginTop`.

3.3. Оновіть усі видимі елементи

Передавайте `colors` формі й привітанню, як у завданні 1. Для шапки та підвалу також додайте властивість `colors: Palette` і застосуйте її; якщо в них уже є тип props, доповніть його, не видаляючи попередні властивості.

Орієнтир для зв'язку кольорів і елементів:

Елемент	Які кольори застосувати
Безпечна оболонка, фон екрана	`background`
Картка, поле введення, шапка або підвал	`surface`
Заголовок, основний текст, введений текст	`text`
Пояснення, `placeholderTextColor`	`muted`
Рамка поля	`border`
Кнопка та її напис	`accent` та `onAccent`
Повідомлення про помилку	`error`

Наприклад, до наявного поля email додайте динамічні кольори, зберігши його `value`, `onChangeText` та параметри клавіатури:

```
style={[
  styles.input,
  {
    color: colors.text,
    backgroundColor: colors.surface,
    borderColor: colors.border,
  },
]}
placeholderTextColor={colors.muted}
```

Це властивості `TextInput`, які вставляють усередину його відкривального тегу, а не окремий JSX-елемент. Для рамки у `styles.input` потрібен `borderWidth`, наприклад `1`.

Якщо у проєкті використовується `StatusBar` із `expo-status-bar`, узгодьте напис системного рядка зі своїм фоном: `style={isDark ? 'light' : 'dark'}`. Для `StatusBar` із `react-native` відповідна властивість називається `barStyle`, а значення — `'light-content'` і `'dark-content'`. Не змішуйте ці два API й не додавайте другий компонент лише заради зміни властивості.

3.4. Перевірте автоматичне перемикання

Запустіть застосунок на телефоні або в емуляторі. У системних налаштуваннях змініть світлу тему на темну, потім поверніть світлу. Перевірте обидва екрани, зокрема введений текст, підказки полів і повідомлення про помилки. Окрема кнопка «Темна тема», якщо ви її додали, не доводить роботу системного режиму.

Зміна теми повинна оновити кольори, але не скидати введений email чи самовільно повертати до форми. Не змінюйте `key` компонентів залежно від теми та не визначайте функції компонентів усередині `App`: це може спричинити їх повторне монтування замість звичайного оновлення.

Додаткові завдання

Адаптуйте привітання до ширини вікна

Імпортуйте `useWindowDimensions` із `react-native`. На верхньому рівні `WelcomeScreen` отримайте ширину та обчисліть варіант компонування:

```
const { width, height, scale, fontScale } = useWindowDimensions();
const wide = width >= 600;
```

У контейнері з логотипом та описом задайте `flexDirection: wide ? 'row' : 'column'`. Блоку тексту надайте `flex: 1`, щоб довгий опис переносився в доступному просторі. Поріг `600` — вибір для цього макета, а не спосіб визначити модель пристрою.

`width` і `height` описують вікно застосунку в логічних одиницях, `scale` — співвідношення фізичних пікселів до логічної одиниці, `fontScale` — системний масштаб шрифту. Можна тимчасово показати ці значення у `Text` для дослідження. Не множте розміри компонентів на `scale` і не множте вручну весь текст на `fontScale`: стандартний `Text` уже враховує системне масштабування, якщо його не вимкнено.

Перевірте інший розмір вікна або поворот пристрою. Якщо проєкт зафіксовано в портретному режимі, для перевірки повороту встановіть `orientation: "default"` у `expo` в `app.json`, застосуйте конфігурацію й перезапустіть застосунок. Порівняйте також результат зі збільшеним системним шрифтом.

Дослідіть мережеве зображення

Додайте окреме `Image` із джерелом `{ uri: 'https://reactnative.dev/img/tiny_logo.png' }`, задайте йому рамку та `resizeMode="contain"`. Це інший спосіб передавання `source`: адреса завантажується під час роботи й потребує мережі.

Реалізуйте стан `useState<'loading' | 'ready' | 'error'>('loading')`. У `onLoadStart` встановлюйте `'loading'`, у `onLoad` — `'ready'`, у `onError` — `'error'`. Під час завантаження показуйте `ActivityIndicator`, а при помилці — зрозуміле повідомлення. Ці компоненти та події не потребують ручного HTTP-запиту через `fetch`.

Перевірте правильну адресу й адресу з помилкою. Подія `onLoadEnd` повідомляє про завершення спроби, але сама по собі не означає успіху. Не прибирайте `Image` з дерева лише тому, що показуєте індикатор: тоді воно не зможе завершити завантаження. Основний локальний логотип залиште незалежним від цієї вправи.

Перевірка результату

У папці проєкту виконайте:

```
npx tsc --noEmit
```

Команда перевіряє типи без створення окремих JavaScript-файлів. Виправте помилки, потім перевірте поведінку на телефоні або в емуляторі. Expo перетворює TypeScript для запуску, але це не замінює окрему перевірку типів.

Дія	Очікуваний результат
Натиснути кнопку з порожніми полями	Є повідомлення про email, привітання не відкривається.
Ввести `student` і будь-який навчальний пароль	Є повідомлення про формат email.
Ввести `student@example.com`, пароль залишити порожнім	Є повідомлення про пароль.
Ввести `student@example.com` і непорожній навчальний пароль	Відкривається привітання з email без крайніх пробілів.
Натиснути кнопку з відкритою клавіатурою	Дія обробляється; після коректного заповнення клавіатура закривається.
Повернутися до форми	Відображаються порожні поля; пароль не залишився у привітанні.
Утримувати кнопку короткий час	Видно стиль натискання; до завершення натискання перехід не виконується.
Відкрити привітання	Логотип повністю видно, він не розтягнутий; довгий вміст прокручується.
Змінити системну тему на кожному екрані	Змінюються фон, текст, поля й повідомлення; введені дані не скидаються лише через зміну теми.
Збільшити системний шрифт	Важливі написи не обрізані, до кнопок можна дістатися прокручуванням.

Кнопки перевіряйте також дотиком, а не лише мишею в браузері. Web-версія допомагає досліджувати компонування, але не замінює перевірку нативної клавіатури, safe areas і системної теми на Android/iOS.

Збережіть вихідні файли, `package.json`, файл блокування залежностей та коротку інструкцію запуску. Підготуйте знімки форми й привітання у світлій і темній темах, а також приклад повідомлення про помилку. `node_modules` до матеріалів роботи додавати не потрібно.

Демонстраційний проєкт

Для ознайомлення використайте готовий приклад **«Кафе “Перерва”»** у матеріалах теми 8: папка `new/Тема 8. Pressable, зображення та хуки/demo/`. Це окремий приклад, а не готова реалізація форми входу Little Lemon: він показує й приховує опис, змінює стан обраного, локально перевіряє форму з email, відображає зображення й адаптує кольори та компонування.

Інструкція запуску й дослідження демо.

Відкрийте термінал у папці демо, виконайте `npm ci`, потім `npm start`. Використовуйте сумісну з ним версію Node.js та Expo Go, зазначену в його `README.md`. Не копіюйте `package.json` демо поверх залежностей свого проєкту.

Рекомендований порядок ознайомлення:

1. `src/ActionButton.tsx`: типи props, callback, `pressed` і недоступна кнопка.
2. `src/PressableExample.tsx`: локальний стан, перевірка введення й умовний вміст.
3. `src/ImagesExample.tsx`: джерела зображення, масштабування й помилка завантаження.
4. `src/useDemoTheme.ts` та `App.tsx`: системна тема, передавання кольорів, безпечна оболонка.
5. `src/HooksExample.tsx`: зміна компонування відповідно до ширини.

Прочитайте коментарі, змініть один напис або стиль натискання та поясніть результат. Кнопки «Світла» й «Темна» у демо задають палітру вручну; для перевірки реакції на системне налаштування виберіть «Авто».

Типові помилки

Проблема	Що перевірити
Дія виконується відразу після відкриття екрана	Передайте `onPress={handleContinue}`, а не результат `handleContinue()`.
Натискання змінює змінну, але не екран	Зберігайте потрібне значення в `useState` й використовуйте setter.
TypeScript не дозволяє записати email замість `null`	Оголосіть `useState<string | null>(null)`.
Компонент не приймає `colors`, `onContinue` або `onExit`	Доповніть тип його props і параметри функції; перевірте імпорт `Palette`.
Кнопка не реагує з відкритою клавіатурою	Перевірте `keyboardShouldPersistTaps="handled"` у `ScrollView` форми.
Логотип не знайдено	Перевірте розташування файла, `../`, розширення і регістр літер.
Логотип обрізано чи спотворено	Задайте рамку та `resizeMode="contain"`; приберіть суперечливі налаштування.
Фон темний, але текст або поля не читаються	Застосуйте палітру також до `Text`, `TextInput`, placeholder і рамок.
Системна тема не оновлюється	Перевірте `automatic`, `expo-system-ui`, відсутність примусового режиму та застосування нативної конфігурації.
Поля очищаються при зміні теми	Не змінюйте `key` за темою; оголошуйте компоненти на рівні модуля, а не всередині `App`.
Помилка про порядок виклику хуків	Перенесіть хуки на верхній рівень функції компонента, до умовних повернень.

Захист роботи

Захист усний, із демонстрацією застосунку. Покажіть некоректне й коректне заповнення форми, перехід до привітання та повернення, локальний логотип і реакцію обох екранів на зміну системної теми. Поясніть структуру файлів, типи props, розташування стану та особливості роботи програми.

Будьте готові відповісти на контрольні питання й внести мінімальні зміни в код: змінити напис чи візуальний стан кнопки, скоригувати повідомлення перевірки, змінити розмір рамки логотипа, додати колір до палітри або змінити світлий запасний варіант для `null`. Поясніть, чому змінюється результат і які компоненти оновлюються.

Контрольні питання

1. Яку роль виконують `Pressable` і вкладений у нього `Text`?
2. Чим передавання функції в `onPress` відрізняється від її виклику в JSX?
3. Чим короткочасний `pressed` відрізняється від стану, створеного через `useState`?
4. Коли доречне оновлення `setValue(previous => ...)`?
5. Чому стан навчального переходу зберігається у спільному батьківському компоненті?
6. Що означають типи `string | null`, `(email: string) => void` та `() => void`?
7. Чому локальна перевірка email і пароля не є справжньою авторизацією?
8. Відносно чого обчислюється шлях у `require` для локального зображення?
9. Чим відрізняються `contain` і `cover`?
10. Звідки імпортуються `useState` та `useColorScheme`? Де дозволено їх викликати?
11. Як у цій роботі обробляється значення `null` від `useColorScheme`?
12. Чому зміни лише `backgroundColor` недостатньо для темної теми?
13. Що відбувається зі станом форми після її демонтування та після звичайного оновлення кольорів?
14. Якщо виконано додаткові завдання: що повертає `useWindowDimensions` і як відрізнити успіх завантаження зображення від завершення спроби?

Корисні джерела

- React Native: Pressable.
- React Native: локальні й мережеві зображення.
- React Native: властивості Image.
- React Native: useColorScheme.
- Expo: системна світла й темна тема.
- React Native: useWindowDimensions.
- React: правила виклику хуків.
- TypeScript: базові типи.